

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1586

**IMPLEMENTACIJA SREDIŠNJEG SUSTAVA ZA
NADZOR I VIZUALIZACIJU KOMUNIKACIJSKIH
OBRAZACA LOKALNE MREŽE**

Ante Čavar

Zagreb, lipanj, 2026.

DIPLOMSKI ZADATAK br. 1586

Pristupnik: **Ante Čavar (0036540817)**
Studij: Računarstvo
Profil: Računarska znanost
Mentor: izv. prof. dr. sc. Stjepan Groš

Zadatak: **Implementacija središnjeg sustava za nadzor i vizualizaciju komunikacijskih obrazaca lokalne mreže**

Opis zadatka:

S porastom broja povezanih uređaja u lokalnim mrežama, vrlo je teško uočiti neželjeni ili maliciozni mrežni promet koji potiče iz lokalne mreže. Standardna mrežna oprema često ne pruža dovoljnu vidljivost u specifičnosti dolaznih i odlaznih mrežnih konekcija, zbog čega je otežano utvrditi s kojim točno vanjskim resursima komuniciraju lokalni uređaji. Detekcija takvog mrežnog prometa ključna je otkrivanje sigurnosnih prijetnji i neovlaštenih pristupa unutar lokalne mreže. U sklopu diplomskog rada potrebno je osmisliti i implementirati mrežnu arhitekturu koja će služiti kao središnja pristupna točka za lokalnu mrežu, omogućavajući usmjeravanje i analizu mrežnog prometa. Potrebno je postaviti i konfigurirati odgovarajuća programska rješenja za inspekciju mrežnih sjednica i detekciju upada kako bi se zabilježili komunikacijski obrasci i identificirali potencijalni indikatori kompromitacije. Na temelju prikupljenih podataka, potrebno je implementirati grafičko sučelje koje će vizualno prikazivati aktivne uređaje u mreži, smjerove njihove komunikacije i evidentirane sigurnosne događaje.

Rok za predaju rada: 26. lipnja 2026.

Zahvaljujem svojoj obitelji što je uvijek bila uz mene.

Sadržaj

1. Uvod	1
2. Teorijska podloga i tehnološki stog	2
2.1. Mrežna infrastruktura i pristupne točke	2
2.2. Sustavi za detekciju upada i analizu prometa	3
2.3. Kontejnerizacija mrežnih servisa	4
2.4. Pohrana vremenskih nizova i vizualizacija	5
3. poglavlje	8
3.1. Pregled sustava	8
3.2. Mrežni sloj	9
3.3. Nadzorni sloj	9
3.4. Tok podataka i pohrana	9
3.5. Prepoznavanje i automatska reakcija	9
4. Implementacija	10
5. Evaluacija	11
6. Testiranje	12
7. Zaključak	13
Literatura	14
Sažetak	16
Abstract	17

Skraćenice	18
-----------------------------	-----------

1. Uvod

Svaki uređaj danas je povezan u neku mrežu. Velika većina tih uređaja pripada jednoj od bezbroj lokalnih mreža. Neke su kućne, neke poslovne, a neke vojne. Činjenica je da su lokalne mreže svugdje oko nas. Sigurnost lokalnih mreža (kao i sigurnost bilo čega drugog) nije pratila razvoj i pristupačnost tih istih mreža. Jednom kada se sigurnosni incidenti krenu nizati se krene buditi i svijest poslovnih i privatnih korisnika.

Kako bi se incidenti (a time i troškovi pogotovo za poslovne korisnike) izbjegli na tržištu se pojavilo mnoštvo programa [1] [2] pa čak i zanimanja (SOC analist). Ta rješenja funkcioniraju za one koji ih mogu platiti. Ako postoji potreba za pravim 24 satnim nadzorom onda firmi nije problem zaposliti (najmanje) 3 SOC analista koji će se izmjenjivati u 8 satnim smjenama. Postoji i mogućnost računalne zaštite koja nije toliko dobra ali i dalje nudi neku razinu zaštite. No ipak problem svih navedenih rješenja je što koštaju.

Cilj ovog rada je ponuditi programsko rješenje osobnim korisnicima i malim firmama kako bi zaštitili svoju lokalnu mrežu unatoč manjoj financijskoj sposobnosti koja nije problem za veće firme zbog toga što su male firme cilj nemalog broja napada čak 43% [3]. U radu su obrađena dva moguća rješenja gore navedenog problema. Korišteni alati sa kojima se radilo je Zeeku kao osnovnom alatu za prikupljanje podataka, nakon toga se ispitivanje provodilo nad 2 para baze podataka i njezinim agregatorom podataka: InfluxDB uz Telegraf kao agentu za prikupljanje podataka te Elasticsearch uz Filebeat. Obje baze podataka na kraju su informacije agregirale u Grafani za vizualizaciju prikupljenih informacija.

Nakon objašnjenja rada i procesa ponuditi će se čitateljima mogući smjerovi za ovo implementacijsko rješenje te koje daljnje korake autor vidi u ovom radu.

2. Teorijska podloga i tehnološki stog

NSM (eng. *Network Security Monitoring*) predstavlja metodologiju prikupljanja, analize i eskalacije indikatora koji ukazuju na maliciozne aktivnosti unutar nekog mrežnog okruženja. Tradicionalni mehanizmi zaštite kao vatrozid i antivirusi se primarno fokusiraju na prevenciju napada izgradnjom barijera. Međutim, kako ističe Bejtlich [4], prevencija s vremenom zakazuje, a napadači kontinuirano pronalaze nove načine za zaobilazanje statičkih obrana. Zato se NSM ne oslanja isključivo na prevenciju nego i na dubinsku vidljivost mrežnog prometa koja omogućava pravovremenu detekciju i reakciju na incidente.

Kako bi se uspostavio funkcionalan NSM sustav, potrebno je integrirati nekoliko različitih klasa alata: od mrežnih senzora za ekstrakciju metapodataka, preko optimiziranih baza podataka za pohranu vremenskih nizova, do grafičkih sučelja za vizualizaciju anomalija. U nastavku ovog poglavlja bit će opisane teorijske osnove tehnologija koje su odabrane kao temelj za izgradnju takvog rješenja.

2.1. Mrežna infrastruktura i pristupne točke

AP (eng. *Access Point*) još znan i kao pristupna točka je uređaj koji se ponaša kao most (eng. *bridge*) i time dopušta bežičnim uređajima poput laptopa, mobitela, IoT senzora da se povežu sa žičanom lokalnom mrežom (LAN). Za razliku od bežičnih rutera koji u sebi kombiniraju preusmjerenje, vatrozid i WiFi funkcionalnosti pristupne točke se fokusiraju u potpunosti na bežično povezivanje (iako se danas ta granice između pristupne točke i rutera sve manje primjećuje).

hostapd je alat na Linuxu s kojim se može jedan od bežičnih mrežnih sučelja sustava pretvoriti u pristupnu točku [5] pod uvjetom da postoji još jedno mrežno sučelje (žičano ili

mrežno) koje će služiti kao izlaz na Internet. Između ostalog `hostapd` nudi mogućnosti:

- nadzora pristupne točke preko IEEE 802.11 standarda
- autentifikacijske protokole poput IEEE 802.1X, WPA, WPA2, i WPA3
- može funkcionirati kao RADIUS Server/Klijent što je posebno bitno firmama i ustanovama

Kako bi uređaji povezani na pristupnu točku mogli funkcionalno komunicirati, potrebno je osigurati mrežnu konfiguraciju i razlučivanje naziva. Ulogu pružanja tih servisa u sustavima s ograničenim resursima često preuzima alat `dnsmasq` [6]. Potrebno je naglasiti da `dnsmasq` ne obavlja izolaciju ili usmjeravanje mrežnog prometa (što je primarni zadatak vatrozida), već isključivo objedinjuje funkcionalnosti DNS prosljeđivača (eng. *DNS forwarder*) i DHCP poslužitelja.

U ulozi DNS prosljeđivača, `dnsmasq` prihvata lokalne DNS upite te ih prosljeđuje vanjskim DNS poslužiteljima kako bi se simbolička imena (domene) prevela u odgovarajuće IP adrese. Kao DHCP poslužitelj (eng. *Dynamic Host Configuration Protocol*), zadužen je za automatsku dodjelu mrežnih parametara. DHCP poslužitelj se brine da se svakom uređaju koji pristupa mreži dinamički dodijeli jedinstvena IP adresa, podmrežna maska i adresa zadannog usmjernika, čime se omogućava neometana komunikacija i izbjegavaju mrežne kolizije.

2.2. Sustavi za detekciju upada i analizu prometa

Zeek je platforma otvorenog koda namijenjena dubinskoj analizi mrežnog prometa. Za razliku od tradicionalnih sustava koji se oslanjaju na bilježenje sirovih mrežnih paketa, Zeek generira kompaktne transakcijske zapisnike (eng. *transaction logs*) visoke preciznosti te omogućava potpuno prilagodljiv izlazni format podataka. Time se analitičarima osiguravaju konkretni i primjenjivi podaci (eng. *actionable data*). Neovisno o veličini okruženja – od malih lokalnih mreža do opsežnih istraživačkih i komercijalnih sustava – Zeek pruža jasan uvid u obrasce korištenja mreže, čime izravno doprinosi nadzoru sigurnosti, reviziji i analizi mrežnih performansi.

Jedna od ključnih prednosti ovog sustava je njegov specifičan, mrežno optimiziran skriptni jezik koji omogućava izradu prilagođenih pravila detekcije i ekstrakciju specifičnih meta-

podataka. Zahvaljujući globalnoj prihvaćenosti i snažnoj podršci zajednice otvorenog koda, Zeek nudi dubinske uvide neophodne za rješavanje najsloženijih sigurnosnih izazova unutar poslovnih, računalnih i *cloud* okruženja.

Najveća prednost Zeeka je mogućnost kreiranja logova ovisno o tipu povezivanja i protokola (`conn.log`, `dhcp.log`, `http.log`, `files.log`, `dns.log` ...). Osim podjele na protokole Zeek nudi i jedinstvenu mogućnost upozorenja (`notice.log`) gdje se korištenjem Zeekovog skriptnog jezika mogu prepoznati specifični tipovi povezivanja ili se uočiti obrasci u ponašanju uređaja u mreži.

Alternative poput Suricata fokusiraju se primarno na detekciju temeljenu na potpisima (eng. *signature-based detection*). Sveobuhvatna rješenja poput Security Oniona, iako funkcionalna, zahtijevaju znatno veće resurse — minimalno 12 GB RAM-a za osnovnu instalaciju [2] [7] — što ih čini neprikladnima za deployment na potrošačkom hardveru. Zeekova prednost leži u fleksibilnoj ekstrakciji metapodataka i prilagodljivom skriptnom jeziku koji omogućava izradu vlastite detekcijske logike uz minimalni memorijski otisak.

2.3. Kontejnerizacija mrežnih servisa

Docker je platforma za razvoj, dijeljenje i pokretanje aplikacija. Omogućava odvajanje aplikacija od infrastrukture [8]. Kako bi to efikasno napravio, Docker koristi kontejnere.

Kontejneri su mala okruženja koja se pokreću na računalu domaćinu te su izolirana od ostalih procesa korištenjem *namespaceova* i *cgroups* tj. kontrolnih grupa. Kontejneri su pokretljive instance slika (eng. *images*) koje možete izraditi, pokrenuti, zaustaviti, premjestiti ili izbrisati pomoću Docker alata. Također su portabilni te se mogu pokrenuti lokalno, u virtualnim strojevima i u oblaku.

Docker se pobrinuo da kontejneri imaju minimalan utjecaj na domaćina te za pokretanje kontejnera nije potrebna instalacija dodatnih ovisnosti na sustavu domaćina. Svaki se kontejner pokreće u izoliranom okruženju te može komunicirati sa ostalim kontejnerima tek ako mu se to eksplicitno dozvoli.

Za orkestraciju više kontejnera koji zajedno čine jednu aplikaciju Docker nudi alat Docker Compose. Konfiguracija cijelog skupa servisa definira se u jednoj *docker-compose.yml*

datoteci koja opisuje svaki servis, njegove ovisnosti, volumene i mrežne postavke. Pokretanje cijelog sustava svodi se na jednu naredbu (`docker compose up -d`), što znatno pojednostavljuje deployment i reproducibilnost okruženja te je glavni razlog zašto je upravo Docker korišten u ovom radu.

2.4. Pohrana vremenskih nizova i vizualizacija

InfluxDB je baza podataka temeljena na Time-Series modelu te koristi Flux jezik za upite. Time-series model baze je optimiziran da sprema, indeksira i izvršava upite nad podacima gdje je vrijeme glavna organizacijska dimenzija. [9] Glavne karakteristike InfluxDB baze podataka:

- indeksiranje temeljeno na vremenu (omogućava brzu obradu velikih količina podataka)
- samo "nadodaj" (eng. *append*) arhitektura (nijedan se zapis ne briše ili uređuje što omogućava pristup povijesti izmjena)
- optimizirano sažimanje i pohrana (koristi kompresiju na temelju vremena te tehnike smanjenja uzorkovanja da lakše rukuje velikim volumenom podataka)
- ugrađene agregacijske funkcije (podrška za vremenski specifične funkcije poput pomičnih prosjeka, percentila i sažetaka temeljenih na intervalima olakšava složenu analizu bez velikih računalnih troškova)

InfluxDB podržava tri moguća jezika upita: SQL, Flux i Influx. U ovom radu korišten je Flux zbog odabrane verzije baze (2.7). Flux je funkcionalni skriptni jezik otvorenog koda dizajniran za upite, analizu i djelovanje na podacima vremenskih serija, prvenstveno služeći kao izvorni jezik upita za InfluxDB 2.x. [10] [11] Za razliku od SQL-sličnog InfluxQL-a koji se koristi u InfluxDB 1.x, Flux je dizajniran da bude kompozibilan i fleksibilan, podržavajući ne samo InfluxDB već i razne SQL baze podataka (npr. PostgreSQL, BigQuery) i formate podataka poput CSV-a i JSON-a. Primjer Flux upita: [12]

```
import "influxdata/influxdb/v1"

option v = {bucket: "my-bucket", org: "my-org"}

from(bucket: v.bucket)
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "cpu")
  |> mean()
```

Kod 2.1. Primjer Flux upita

Upit 2.1 definira "kantu" informacija te se iz tog *bucketa* redom povlače informacije na sljedeći način: svi logovi iz prethodnih sat vremena (`start: -1h`), koji u sebi sadrže informaciju o CPU-u (`filter(fn: (r) => r._measurement == "cpu")`) te iz tih informacija izvuci srednju vrijednost (`mean()`). Drugim riječima ovaj upit će prikupiti informacije o CPU-u u zadnjih sat vremena te će prikazati njihovu srednju vrijednost.

Telegraf je alat za prikupljanje podataka otvorenog koda koji je razvio InfluxData, a koji prikuplja, obrađuje i agregira metrike, logove i druge podatke iz različitih izvora. Služi kao primarni sloj za unos podataka za InfluxDB, bazu podataka vremenskih serija, koristeći arhitekturu temeljenu na dodacima s preko 300 ulaznih i izlaznih dodataka za prikupljanje podataka iz sustava, senzora i aplikacija te njihov izravan zapis u InfluxDB. [13] [14] Neke od karakteristika Telegrafa su:

- Arhitektura temeljena na dodacima (*pluginovi*) (korisnik konfigurira ulaze i izlaze u `telegraf.conf` što omogućuje fleksibilno upravljanje tokom podataka bez konkretnog znanja programiranja)
- Napisan u jeziku GO (jedna binarna datoteka sa mogućnošću obrade ogromne količine podataka sa više uređaja)
- predprocesiranje podatka (može očistiti, filtrirati i transformirati podatke prije nego što stignu do baze podataka)

Uz InfluxDB i Telegraf, kao alternativa ispitan je i Elasticsearch uz Filebeat kao agentom za prikupljanje podataka.

Elasticsearch je distribuirani sustav za pretraživanje i analizu podataka otvorenog koda temeljen na Apache Lucene biblioteci. [15] Za razliku od InfluxDB-a koji optimizira po-

hranu vremenskih serija, Elasticsearch tretira svaki zapis kao JSON dokument koji se indeksira kroz invertirani indeks (eng. *inverted index*) — struktura podataka koja omogućava brzu pretragu po sadržaju polja. Ta arhitektura čini Elasticsearch pogodnim za *full-text* pretraživanje i korelaciju podataka iz heterogenih izvora, no dolazi s većim memorijskim i prostornim otiskom u usporedbi s InfluxDB-om što će biti detaljnije prikazano u poglavlju evaluacije.

Kao što Telegraf služi kao agent za unos podataka u InfluxDB, Filebeat preuzima analognu ulogu u Elasticsearch ekosustavu. [16] Filebeat je lagani agent koji prati zadane datoteke zapisnika i prosljeđuje nove zapise u Elasticsearch. U kontekstu ovog rada, Filebeat čita Zeekove JSON logove iz dijeljenog volumena i indeksira ih u Elasticsearch bez transformacije podataka.

Grafana je platforma otvorenog koda za vizualizaciju i analizu podataka koja podržava više od pedeset izvora podataka, uključujući InfluxDB i Elasticsearch. [17] Dashboardi se konfiguriraju kroz grafičko sučelje i mogu se izvesti kao JSON datoteke te pohraniti u repozitorij, što omogućava automatsko učitavanje pri pokretanju sustava (eng. *provisioning*). U ovom radu Grafana služi kao jedinstveno vizualizacijsko sučelje za oba ispitana backend rješenja.

3. poglavlje

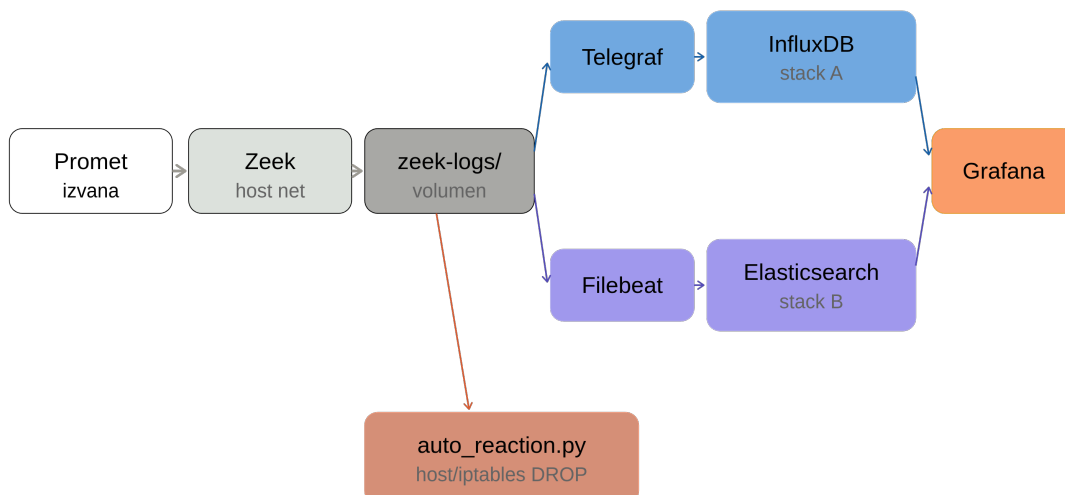
Arhitektura U ovom se poglavlju opisuje arhitektura predloženog (kasnije i implementiranog) sustava. Opisuje se način na koji je lokalna mreža usmjerena kroz pristupnu točku, gdje se i kako prikuplja mrežni promet te kako se prikupljeni podatci pohranjuju, vizualiziraju te u konačnici kako se s tim podacima rukuje kako bi se postigla automatska detekcija. Naglasak je na odlukama o dizajnu i njihovim opravdanjima, dok se konkretna implementacija obrađuje kasnije.

3.1. Pregled sustava

Sustav je zamišljen da pretvori uređaj s mrežnim mogućnostima u jedinstvenu točku kroz koju prolazi sav promet lokalne mreže, čime se ostvaruje potpuna vidljivost komunikacije između lokalnih uređaja i vanjskih odredišta. Taj uređaj istovremeno obavlja funkciju usmjernika (eng. *gateway*), pristupne točke, mrežnog senzora i vatrozida.

Slika 3.1. prikazuje tok podataka kroz sustav. Sav mrežni promet prolazi kroz pristupnu točku gdje ga Zeek pasivno analizira i pohranjuje u obliku strukturiranih JSON zapisa u dijeljeni direktorij. Iz tog se direktorija podatci paralelno "šalju" u dva odvojena podsustava za pohranu. Prvi sustav čine vremenski orijentirana baza InfluxDB posredstvom podatkovnog agregatora Telegrafa. Drugi sustav čine dokumentno orijentiranu bazu Elasticsearch posredstvom agenta zvanog Filebeat. Oba podsustava kao zajedničko vizualizacijsko sučelje koriste Grafanu. Usporedno postojanje dvaju podsustava ne predstavlja produkcijsku konfiguraciju, već služi za njihovu usporedbu opisanu u kasnijim poglavljima. U stvarnoj bi se primjeni odabrao jedan od njih ovisno o potrebama i mogućnostima.

Svi navedeni servisi izvode se u Docker kontejnerima, čime se postiže ponovljivost i prenosivost okruženja te jednostavnost pri postavljanju [8]. Jedina je iznimka skripta za

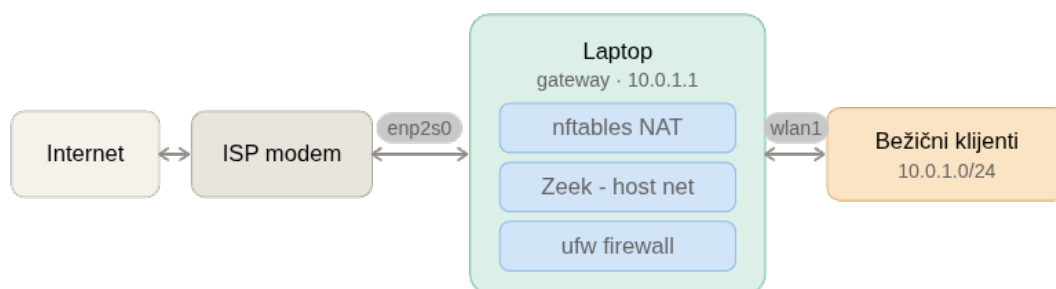


Slika 3.1. Prikaz toka podataka kroz sustav

automatsku reakciju, koja se izvodi izravno na domaćinu iz kasnije objašnjenih razloga.

3.2. Mrežni sloj

Mrežna topologija sustava prikazana je na slici 3.2.



Slika 3.2. Mrežna topologija. Prijenosno računalo objedinjuje uloge rutera, pristupne točke, senzora i vatrozida

3.3. Nadzorni sloj

3.4. Tok podataka i pohrana

3.5. Prepoznavanje i automatska reakcija

4. Implementacija

5. Evaluacija

6. Testiranje

7. Zaključak

Literatura

- [1] Splunk Inc., “Splunk enterprise documentation”, 2025., pristupljeno: 20.5.2026. [Mrežno]. Adresa: <https://docs.splunk.com/Documentation/Splunk>
- [2] Security Onion Solutions, “Security onion — intrusion detection, enterprise security monitoring, and log management”, 2025., pristupljeno: 20.5.2026. [Mrežno]. Adresa: <https://securityonionsolutions.com/software>
- [3] Total Assure, “Cyber attacks on small businesses statistics 2026”, Total Assure, teh. izv., 2026., pristupljeno: 22.5.2026. [Mrežno]. Adresa: <https://www.totalassure.com/blog/cyber-attacks-on-small-businesses-statistics>
- [4] R. Bejtlich, *The Practice of Network Security Monitoring*. No Starch Press, 2013.
- [5] J. Malinen, “hostapd: Ieee 802.11 ap, ieee 802.1x/wpa/wpa2/eap/radius authenticator”, 2024., pristupljeno: 13.3.2026. [Mrežno]. Adresa: <https://w1.fi/hostapd/>
- [6] S. Kelley, “dnsmasq — a lightweight dhcp and caching dns server”, 2024., pristupljeno: 13.3.2026. [Mrežno]. Adresa: <https://dnsmasq.org>
- [7] Security Onion Solutions, pristupljeno: 21.5.2026. [Mrežno]. Adresa: <https://docs.securityonion.net/en/2.4/hardware.html>
- [8] Docker Inc., “Docker documentation”, 2025., pristupljeno: 1.6.2026. [Mrežno]. Adresa: <https://docs.docker.com>
- [9] InfluxData, “Influxdb 2.x documentation”, 2025., pristupljeno: 1.6.2026. [Mrežno]. Adresa: <https://docs.influxdata.com/influxdb/v2/>

- [10] —, “Get started with flux”, 2024., pristupljeno: 12.4.2026. [Mrežno]. Adresa: <https://docs.influxdata.com/flux/v0/get-started/>
- [11] —, “Influxdb: Flux basics”, 2022., youTube. Pristupljeno: 12.4.2026. [Mrežno]. Adresa: <https://www.youtube.com/watch?v=LbQCYNGcWUU&t=1s>
- [12] —, “Flux query language and influxdb basics”, 2024., pristupljeno: 12.4.2026. [Mrežno]. Adresa: <https://docs.influxdata.com/influxdb/v2/query-data/get-started/>
- [13] —, “Telegraf documentation”, 2025., pristupljeno: 1.6.2026. [Mrežno]. Adresa: <https://docs.influxdata.com/telegraf/v1/>
- [14] —, “Influxdb: Intro to telegraf”, 2022., youTube. Pristupljeno: 12.4.2026. [Mrežno]. Adresa: <https://www.youtube.com/watch?v=sF2inT5tITg>
- [15] Elastic, “Elasticsearch documentation”, 2025., pristupljeno: 20.5.2026. [Mrežno]. Adresa: <https://www.elastic.co/docs/solutions/search>
- [16] —, “Filebeat reference”, 2025., pristupljeno: 29.5.2026. [Mrežno]. Adresa: <https://www.elastic.co/docs/reference/beats/filebeat>
- [17] Grafana Labs, “Grafana documentation”, 2025., pristupljeno: 10.5.2026. [Mrežno]. Adresa: <https://grafana.com/docs/grafana/latest/>

Sažetak

Implementacija središnjeg sustava za nadzor i vizualizaciju komunikacijskih obrazaca lokalne mreže

Ante Čavar

None

Ključne riječi: None

Abstract

Implementation of a centralized system for monitoring and visualizing local network communication patterns

Ante Čavar

None

Keywords: None

Skraćenice